

psy_detectives at SemEval-2026 Task 10: PsyCoMark – Psycholinguistic ConspiracyMarker Extraction and Detection

Roxana Carabaş, Anamaria Persida Nacu, Isac Lucian-Constantin
Faculty of Computer Science
“Alexandru Ioan Cuza” University of Iaşi
Iaşi, Romania

Abstract—This paper presents baseline systems developed for PsyCoMark (SemEval-2026 Task 10), which addresses conspiracy detection supported by psycholinguistic marker evidence. We implement three components: (i) an interpretable, marker-driven module that derives a token-to-category lexicon from gold marker spans and computes LIWC-style category ratios; (ii) supervised transformer baselines for binary document-level conspiracy detection (Yes/No) using RoBERTa/DeBERTa-family encoders; and (iii) a zero-shot prompting baseline using TinyLlama-1.1B for three-way classification (Yes/No/Can’t tell).

We also experimented with a lightweight heuristic approach intended to relabel ambiguous instances using marker-derived features, but it did not yield improvements and is therefore not used in our final submissions. On the official development split (dev_rehydrated + dev_public), the best model is microsoft/deberta-v3-large with 0.8339 weighted F1. On the competition test set, the same model achieves 0.75 weighted F1, outperforming the other evaluated transformer variants. TinyLlama achieves 0.34 accuracy on a 500-instance sample and shows a tendency toward predicting “Yes”, highlighting limitations of zero-shot inference with compact instruction-tuned models.

Index Terms—conspiracy detection, psycholinguistics, marker evidence, transformers, prompting, lexicon-based features

I. INTRODUCTION

Conspiracy-related discourse is prevalent in online communities and has been associated with misinformation propagation, social polarization, and declining institutional trust. Automatic conspiracy detection can support monitoring and analysis; however, purely predictive models are frequently criticized for limited interpretability, particularly when the linguistic evidence driving a decision is not explicit. PsyCoMark addresses this limitation by coupling document-level conspiracy labels with span-level psycholinguistic marker annotations belonging to a small set of interpretable categories (e.g., Actor, Evidence, Threat).

This work focuses on reproducible baselines suitable for a shared-task and course context. Specifically, we: (i) construct marker-informed lexical resources and LIWC-style ratio features from annotated spans; (ii) establish supervised transformer baselines for binary conspiracy detection; (iii) report an exploratory heuristic relabeling attempt that did not help; and (iv) include a TinyLlama zero-shot baseline retained from another project for reference.

II. RELATED WORK

Transformer encoders are widely used for text classification, including misinformation and conspiracy-related detection tasks. Fine-tuning pretrained language models such as BERT and its successors remains a standard approach when labeled training data is available. Prompt-based inference using instruction-tuned language models has gained popularity when training data is limited, but smaller models often exhibit sensitivity to prompt design and label bias.

Lexicon-based approaches and LIWC-style category features remain relevant for interpretability. PsyCoMark’s marker annotations directly support such methods by enabling transparent, human-aligned feature construction.

III. DATASET AND TASK SETUP

A. Labels

The rehydrated dataset contains three document-level labels:

- **Yes:** conspiratorial reasoning is present,
- **No:** conspiratorial reasoning is absent,
- **Can’t tell:** evidence is ambiguous or insufficient.

For supervised transformer training, we restrict to a binary setting (Yes/No). The TinyLlama baseline retains all three labels.

B. Marker Schema

Span-level markers belong to six psycholinguistic categories: *Actor*, *Action*, *Victim*, *Evidence*, *Threat*, *Effect*.

C. Rehydration

Due to data redaction, we follow the official rehydration procedure to obtain train_rehydrated.jsonl, dev_rehydrated.jsonl, and test_rehydrated.jsonl.

IV. METHODS

A. Preprocessing

We apply minimal normalization to preserve linguistic signal:

- URL masking with a special token (e.g., $\langle URL \rangle$),
- whitespace normalization,
- sentence segmentation using spaCy (en_core_web_sm).

B. Candidate Sentence Selection

To support downstream evidence inspection and annotation, we extract candidate sentences that are likely to contain conspiratorial reasoning cues. A sentence is retained if it satisfies at least one criterion:

- 1) it contains modal verb lemmas (e.g., *must*, *should*, *could*),
- 2) it ends with a question mark,
- 3) it contains generic agent-related keywords (e.g., *they*, *elites*, *government*).

This component is primarily intended for analysis and interpretability; the transformer baselines are trained on full-document text.

C. Marker Lexicon and LIWC-style Features

Using gold marker spans, we build a token-to-category lexicon by frequency counting and dominant-category assignment. From this lexicon, we compute document-level LIWC-style features: per-category token counts and normalized ratios (counts divided by the number of non-punctuation tokens). These features provide a transparent, human-aligned representation of the psycholinguistic evidence available in PsyCoMark.

D. Heuristic Relabeling Attempt (Not Used)

We experimented with a lightweight heuristic method intended to reduce the impact of ambiguous labels. Using only confidently labeled instances (Yes/No), we trained a logistic regression classifier on marker-derived features (e.g., category ratios and a “marker diversity” feature indicating how many categories appear in a document). We then attempted to relabel Cant instances based on predicted probabilities using conservative thresholds.

In practice, this approach did not improve downstream performance and sometimes introduced additional noise, so we do not use it in our final systems.

E. Baselines

Supervised transformer fine-tuning (binary). We fine-tune pretrained transformer encoders for binary sequence classification (No/Yes). Documents are tokenized with the model tokenizer, truncated to a maximum length, and trained using the HuggingFace Trainer. Optimization uses AdamW with weight decay and a cosine learning-rate schedule with a warmup ratio. We use gradient accumulation to fit larger models and enable mixed precision (FP16) when CUDA is available.

TinyLlama (zero-shot, three-way). We retain the TinyLlama baseline (TinyLlama-1.1B-Chat-v1.0) from another project as a reference point for prompt-based three-way classification (Yes/No/Cant tell). It is evaluated in a zero-shot setting with a constrained instruction prompt and greedy decoding, without additional training.

V. EXPERIMENTS

A. Data Splits

For supervised training, labels are mapped as No→0 and Yes→1, and Cant is excluded. Model selection is performed on the official development split: text from dev_rehydrated.jsonl paired with labels from dev_public.jsonl. Unless stated otherwise, all weighted F1 results reported for transformer models are computed on this dev dataset.

B. Training and Search Setup

Small/medium models. For DistilRoBERTa, RoBERTa-base, DeBERTa-v3-small, and DeBERTa-v3-base, we used Optuna with a TPE sampler to search over maximum sequence length, learning rate, weight decay, warmup ratio, batch size, gradient accumulation steps, epochs, and gradient checkpointing. Each small/medium model was run with 5 trials.

Large models. For roberta-large and deberta-v3-large, full sweeps were computationally expensive. To train in a reasonable amount of time, we used a custom configuration and changed evaluation and checkpointing to step-based scheduling: eval_strategy='steps' and save_strategy='steps' (instead of epoch-based). We also enabled gradient checkpointing, used a small per-device batch size with gradient accumulation, and capped training with max_steps. In this regime, each large model was run once with the fixed configuration.

C. Metrics

For transformer models we report Accuracy and weighted Precision, Recall, and F1 on the dev dataset. We also report weighted F1 on the competition test dataset for the selected best model.

VI. RESULTS

A. Dev Results (Best per Model)

Table I summarizes the best dev weighted F1 achieved by each evaluated transformer model. The best dev model is microsoft/deberta-v3-large.

Model	Alias	Dev Weighted F1
DistilRoBERTa	distilroberta	0.7614
DeBERTa-v3 Small	deberta_small	0.7581
RoBERTa Base	roberta_base	0.7465
DeBERTa-v3 Base	deberta_base	0.7496
RoBERTa Large	roberta_large	0.7640
DeBERTa-v3 Large	deberta_large	0.8339

TABLE I
BEST WEIGHTED F1 ON THE DEV DATASET. LARGE MODELS USE A
TIME-BOUNDED, STEP-BASED TRAINING CONFIGURATION.

B. Best Hyperparameter Configurations

We report the best configurations found for the small/medium models (Optuna search) and the fixed configuration used for the large models (time-bounded run).

Alias	L	lr	wd	warm	bs	ga	ep	gc
distilroberta	320	5.88e-06	0.0366	0.1075	8	2	6	False
deberta_small	256	4.30e-06	0.0156	0.0399	8	1	3	True
roberta_base	320	5.88e-06	0.0366	0.1075	8	2	6	False
deberta_base	256	4.30e-06	0.0156	0.0399	8	1	3	True

TABLE II

BEST CONFIGS FOR SMALL/MEDIUM MODELS (OPTUNA).

L =MAX_LENGTH, LR =LEARNING_RATE, WD =WEIGHT_DECAY,
WARM=WARMUP_RATIO, BS =PER_DEVICE_BATCH, GA =GRAD_ACCUM,
 EP =EPOCHS, GC =GRADIENT_CHECKPOINTING.

Alias	L	steps	lr	wd	warm	bs	ga	ep	gc
roberta_large	192	400	1e-05	0.01	0.06	2	16	1	True
deberta_large	192	400	1e-05	0.01	0.06	2	16	1	True

TABLE III

FIXED, TIME-BOUNDED CONFIGS FOR LARGE MODELS. TRAINING USED
 $EVAL_STRATEGY='STEPS'$ AND $SAVE_STRATEGY='STEPS'$ TO
SUPPORT A STEP BUDGET.

C. Test Performance

Using the best model on dev, microsoft/deberta-v3-large, we obtain a weighted F1 of **0.75** on the competition test dataset. This makes DeBERTa-large the best model in our experiments on both dev and test.

D. TinyLlama Baseline

TinyLlama achieves 0.34 accuracy on a 500-instance sample and shows a strong bias toward predicting the “Yes” label, highlighting limitations of zero-shot inference with compact instruction-tuned models.

VII. CONCLUSION

We presented reproducible baselines for PsyCoMark, combining an interpretable marker-informed feature pipeline with supervised transformer fine-tuning, and a TinyLlama zero-shot reference baseline. The best-performing system in our experiments is DeBERTa-v3-large, which achieves strong results on both dev and test. Future work will explore explicit marker extraction, integrating marker-derived features into neural models, and supervised three-way classification including the Cant label.

REFERENCES

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *NAACL-HLT*, 2019.
- [2] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” *EMNLP: System Demonstrations*, 2020.
- [3] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [4] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” *arXiv preprint arXiv:1910.01108*, 2019.
- [5] P. He, X. Liu, J. Gao, and W. Chen, “DeBERTa: Decoding-enhanced BERT with Disentangled Attention,” *International Conference on Learning Representations (ICLR)*, 2021.
- [6] P. He, X. Liu, J. Gao, and W. Chen, “DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with Gradient-Disentangled Embedding Sharing,” *arXiv preprint arXiv:2111.09543*, 2021.
- [7] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework,” *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2019.
- [8] M. Honnibal et al., “spaCy: Industrial-strength Natural Language Processing in Python,” 2020.
- [9] TinyLlama Team, “TinyLlama-1.1B-Chat-v1.0,” 2024.
- [10] PsyCoMark Task Organizers, “SemEval-2026 Task 10: PsyCoMark,” 2026.